

Exclusión Mutua Distribuida Cliente/Servidor

Algoritmos Distribuidos



UNIVERSIDAD AUTÓNOMA METROPOLITANA
UNIDAD IZTAPALAPA

Problema: Exclusión mutua

- Considere un sistema de n procesos
 - $P_1, P_2, \dots, P_n$
- Los segmentos de código en los cuales un proceso manipula un recurso compartido de uso exclusivo se denominan *secciones críticas*.
- Para no tener resultados erróneos, la sección crítica se debe ejecutar en *exclusión mutua*: es decir, sólo un proceso a la vez está ejecutando su sección crítica.

Problema: Exclusión mutua

- **Suponiendo que:**
 - Un proceso ejecuta su sección crítica en un tiempo finito
 - Si un proceso se detiene fuera de la sección crítica no interfiere con los otros procesos
 - No se tienen restricciones ni conocimiento acerca de las velocidades relativas de los procesos.
- **Una solución al problema de la sección crítica debe cumplir los siguientes requisitos**
 - Exclusión mutua: Si P_i está ejecutando su sección crítica, ningún otro proceso está ejecutando su sección crítica
 - Espera limitada: Nadie debe esperar indefinidamente el entrar en la sección crítica.
 - Progreso: Cuando nadie está ejecutando la sección crítica y un proceso solicita la entrada, se le debe conceder sin retardo.

Problema: Exclusión mutua

- En toda solución pesimista hay una fase de entrada en donde se solicita el permiso de ejecución y una fase de salida en donde se notifica al sistema la finalización de la sección crítica.

ADQUIERE DERECHO DE ENTRADA
EJECUTA SECCIÓN CRÍTICA
NOTIFICA TERMINACIÓN

Solución cliente/servidor

- Un nodo en particular funge como servidor.
- Cuando un nodo requiere usar el recurso compartido envía un mensaje al servidor y espera por la respuesta. Cuando termina de usar el recurso, envía un mensaje de liberación.
- Cuando el servidor recibe una solicitud, si el recurso está libre, contesta favorablemente, en otro caso encola la solicitud. Cuando el servidor recibe una liberación, si hay solicitudes encoladas, envía un Ok al nodo al frente de la cola, en otro caso, marca el recurso como libre.



Modelos

- Se asume conectividad bidireccional entre el servidor y todos los clientes
- Los demás modelos estipulados.



Estado

- **Estado Servidor:**
 - Estado E {LIBRE, OCUPADO}. LIBRE cuando ningún nodo lo está usando y OCUPADO en otro caso. Valor inicial: LIBRE.
 - Q: cola de solicitudes pendientes. Inicialmente está vacía.
- **Mensajes**
 - SOLICITUD: lo envía el cliente al servidor
 - OK: lo envía el servidor al cliente cuando le otorga el derecho de usar el recurso
 - LIBERA: lo envía el cliente al servidor cuando ha terminado de usar el recurso.



Algoritmo Servidor

Al recibir SOLICITUD de Pj

SI estado = LIBRE ENTONCES

estado = OCUPADO

Envia OK a Pj

OTRO

Encola (Q,Pj)

Al recibir LIBERA de Pj

SI EsVacía(Q) ENTONCES

Estado = LIBRE

OTRO

ENVIA OK a Frente(Q)

Cola = resto(Q)

Algoritmo cliente

DLOCK()
 Sección crítica
DUNLOCK()



APLICACIÓN

DLOCK()
 Envía SOLICITUD al Servidor
 Espera OK del Servidor

DUNLOCK()
 Envía LIBERA al Servidor

Propiedades y costos

- ✓ Garantiza la exclusión mutua
- ✓ No presenta inanición (espera limitada)
- ✓ Progreso

Costo en mensajes por acceso a la SC: 3

Tiempo entre la terminación de una solicitud y que otro nodo pueda entrar a su sección crítica: tiempo en el que se transmiten dos mensajes.



Ventajas y desventajas

Ventaja: Simple de implementar

Desventajas:

El servidor es un cuello de botella

El servidor es un punto único de falla

